Technical Report: Lau Drone Farming Automation

Farm Automation Codebase

May 29, 2026

Abstract

This report documents a Lau-based drone farming automation system implemented across five files: `Automation5.lau`, `Config.laum`, `FarmingOperations.laum`, `Movement.laum`, and `Commands.laum`. The system automates planting, crop processing, fruit harvesting, seed purchasing, gear purchasing, and item selling under the constraints of the Lau runtime, including limited cache keys, parser sensitivity, paste-size limits, and asynchronous drone command behavior.

The final design favors a compact state machine, event-driven market handling, garden-driven traversal, explicit movement synchronization, and a lightweight service hook for repeated background work. The most important algorithmic shift was moving away from a full snake-pattern traversal and toward direct iteration over `garden.getGardenPositions()`, reducing the number of tiles that must be evaluated during active sweeps.

1 Project Context

The codebase controls an in-game farming drone using the Lau scripting runtime. The main script runs with the `--!ndrone` pragma, which allows non-blocking drone commands. In this mode, issuing a physical command while the drone is busy may cause the command to be ignored. The automation therefore must explicitly wait for `Enum.DroneStatus.Sleep` before issuing movement, crop, or plant actions that depend on the previous physical action.

The implementation is constrained by five practical limits:

- The system must remain split across exactly five files.
- Each file must stay below the paste and character limits of the Lau environment.
- The runtime cache allows only five keys per player.
- The Lau parser is sensitive to some compact syntax, so stable multiline control flow is preferred.
- Background work must be cooperative, since direct infinite loops in command handlers can block other logic.

2 File-Level Architecture

The modules export positional tables rather than relying on named module fields. This is less readable than named access, but it is more reliable in the observed Lau runtime. Important slots include:

File	Responsibility
`Automation5.lau`	Entrypoint, module wiring, market event handlers, cache-backed state machine, and main loop.
`Config.laum`	Static configuration, seed lists, field state tables, coordinate maps, gear statistics, and state-key construction.
`Movement.laum`	Synchronized wrapper around `droneV2.goto` with duplicate-coordinate skipping.
`FarmingOperations.laum`	Field operations: seed inventory lookup, seed buying, garden scan, garden-driven sweep, crop handling, plant pass, and service hook integration.
`Commands.laum`	Chat command handling, fruit harvesting service, autosell service, seed switching, scan forcing, and summary gear reporting.

Table 1: Module responsibilities.

- `farming[1]`: initialize farming operations.
- `farming[3]`: buy configured seeds.
- `farming[6]`: active sweep.
- `farming[9]`: garden scan.
- `farming[10]`: seed inventory count.
- `farming[12]`: one-tile planting pass.
- `farming[13]`: known empty tile count.
- `farming[16]`: service hook setter.
- `commands[2]`: background service step.

3 State Machine

The main loop uses a compact cache-backed state machine. The cache keys are:

Key	Meaning
`S`	Current farming state: `F`, `A`, or `P`.
`SS`	Seed-stock event status.
`GS`	Gear-stock event status.
`C`	Command channel, currently used for `SCAN`.
`M`	Human-readable mode marker.

The state values are:

- `F`: full refresh state. It calls `gardenScan()` and then runs an active sweep.
- `A`: active sweep state. It iterates over currently occupied garden positions.

- `P`: planting state. It plants one known-empty tile per pass.

The design avoids adding cache keys because the runtime limit is five keys per player. Transient state, such as service flags and last autosell time, is stored in module-local variables instead.

4 Traversal Algorithm

4.1 Previous Snake Traversal

Earlier versions used a snake traversal over the configured grid:

$$T_{\text{snake}} = N^2$$

For the current grid size of $N = 9$, a full snake sweep evaluates 81 tiles. This approach is robust because it can discover empty tiles and unexpected field changes, but it spends operations on empty or irrelevant positions.

4.2 Current Garden-Driven Sweep

The current active sweep calls:

```
varol gp=garden.getGardenPositions()
for k,n inpairs(gp) do
    varol cd=cByKey[k]
    if cd then
        fld[k]="planted"
        varol tileWorked,nextBudget=doTile(cd[1],cd[2],sB,tO)
        ...
    end
end
```

The traversal cost becomes:

$$T_{\text{garden}} = P$$

where P is the number of occupied plant positions returned by `garden.getGardenPositions()`. For a sparse field, this avoids the majority of coordinate checks and movement decisions. The lookup table `COORD_BY_KEY` maps string keys such as `"-1,2"` back to coordinates without string parsing.

5 Tile Processing Algorithm

The `doTile` function is intentionally narrow:

1. Move to the target coordinate through `Movement.laum`.
2. Read the plant under the drone with `drone.getPlant()`.
3. If the tile is empty, update field state.
4. If the plant bears fruit, mark it planted and leave fruit harvesting to the service loop.

5. If the plant is a crop, inspect growth percent.
6. If a crop is near-ready, wait for maturity.
7. Crop mature plants and replant if seed budget is available.

This separation is important. Fruit harvesting is handled by a service loop, while crop processing remains in `doTile` because crop plants require the explicit `drone.crop()` and optional replant sequence.

6 Fruit Harvesting Service

Fruit harvesting was moved out of the active tile path and into `Commands.laum`. The service is enabled with the `init` or `harv` chat command and disabled with `init off` or `harv off`. The service step is called from the main loop and from longer farming waits through the farming service hook.

The core harvesting condition is:

```
if drone.status() == Enum.DroneStatus.Sleep then
    varol p = drone.getPlant()
    if p AND p.HasFruit AND drone.canHarvest() then
        drone.harvest()
        return true
    end
end
```

This design avoids waiting inside `doTile` for fruit maturity. It also keeps fruit harvesting opportunistic: if the drone is idle over a mature fruit plant, the service can harvest without requiring the active sweep to perform a direct harvest branch.

7 Planting Strategy

Full scan refreshes the field model:

1. Mark all configured field coordinates as `"empty"`.
2. Call `garden.getGardenPositions()`.
3. Mark returned coordinates as `"planted"`.

The planting pass then searches the known coordinate list for an empty tile. Current behavior plants one tile and returns immediately. This makes planting incremental and prevents a single state from monopolizing the main loop.

8 Market Automation

Seed and gear buying are event-driven. The script connects to:

- `market.changedSeedStock`

- `market.changedGearStock`

When seeds refresh, the script buys configured seeds and pushes the state machine toward planting. When gears refresh, it buys configured gear types and updates stock statistics. Watering cans were removed from the buy path after watering was disabled.

Autosell is implemented as a service in `Commands.laum`. When enabled by `autosell on`, the service periodically calls `market.sellAllItem()` using `task.clock()` and the configured interval.

9 Movement Technique

`Movement.laum` wraps `droneV2.goto`. It performs two important safety checks:

- Wait until the drone is sleeping before issuing movement.
- Skip movement if the requested coordinate matches the last commanded coordinate.

The direct `goto` approach is faster than stepwise cardinal movement, and duplicate-coordinate skipping prevents redundant commands when the drone is already at the desired location.

10 Problems and Solutions

Problem	Cause	Solution
Cache limit exceeded	More than five cache keys were attempted.	Kept only `S`, `SS`, `GS`, `C`, and `M`; moved other state into module locals.
Chat-started loop blocked work	A long-running chat callback can prevent other logic from advancing.	Converted fruit harvesting and autosell into a service step called from the main loop and farming waits.
Fruit tile time stayed high	`doTile` spent time moving, inspecting, checking fruit percent, and waiting or harvesting directly.	Removed fruit waiting from `doTile`; fruit harvest now runs opportunistically through `harvStep`.
Snake traversal overhead	The snake pattern evaluated every coordinate, even when only occupied plants mattered.	Replaced active sweep traversal with direct iteration over `garden.getGardenPositions()`.
Parser sensitivity	Named table keys and undeclared symbols caused runtime errors in some cases.	Preferred positional exports and declared new variables with `varol`.
Autosell ran once but did not repeat	Autosell was originally tied to command-loop behavior that did not continue while farming work occupied execution.	Integrated autosell into `commands[2]()` and called that service from the main loop and farming wait paths.

11 Performance Evaluation

The optimization process showed that not all bottlenecks were caused by ordinary branch overhead. Several measured and observed outcomes guided the final design:

- Removing watering reduced harvest tile time from roughly 2.8 seconds to about 2.1 seconds.
- Cleaning up `doTile` reduced tile time further during intermediate tests.
- Micro-optimizing movement polling had limited impact, suggesting the remaining cost was dominated by physical movement and runtime API calls.
- Direct fruit-harvest fast paths reduced some overhead but did not eliminate the fundamental cost of tile visits.
- The largest structural improvement was reducing how many tiles are visited, not only reducing the work done at each visited tile.

The final garden-driven active sweep is therefore algorithmically more important than a smaller `doTile` branch. If $P \ll N^2$, replacing the snake traversal with garden iteration reduces both interpreter operations and unnecessary movement decisions.

12 Effectiveness

The current design is effective for a field dominated by occupied plants and fruit-bearing trees:

- It reduces traversal work by targeting occupied garden positions.
- It preserves crop support through explicit crop maturity checks and replanting.
- It keeps fruit harvesting responsive through the service loop.
- It reacts immediately to market restocks through event handlers.
- It keeps long-running services cooperative instead of blocking command callbacks.
- It stays within the five-cache-key limit and the five-file structure.

The system is less optimized for cases where garden metadata is incomplete or where many empty tiles must be planted quickly. The one-tile plant pass is deliberately conservative, trading planting throughput for main-loop responsiveness and reduced blocking.

13 Limitations

- `garden.getGardenPositions()` returns coordinate keys and plant names, but not fruit percent. This limits pre-move filtering for fruit readiness.
- `drone.getPlant()` and `drone.canHarvest()` still require the drone to be physically positioned over a plant.
- Crop plants still need growth polling when near-ready, because crop harvesting uses a different action path from fruit harvesting.

- Numeric module indexing is compact and runtime-safe, but harder to maintain than named exports.
- The design assumes the `init` service is enabled when fruit harvesting is desired.

14 Future Work

Possible next improvements include:

- Add a lightweight command to report service state without restoring the older verbose status command.
- Remove remaining startup debug prints if paste size or console noise becomes important.
- Consider planting more than one empty tile per `P` state only if planting throughput becomes a real bottleneck.
- Investigate whether seed-specific garden APIs, such as `garden.getPlantEnum`, can provide richer metadata cheaply enough to improve targeting.
- Replace numeric slots with named exports only if the runtime is confirmed to handle named module tables reliably.

15 Conclusion

The codebase evolved from a broad, defensive grid traversal into a compact garden-driven automation system. The most effective changes were not traditional hardware-style micro-optimizations, but runtime-aware reductions in interpreter work: fewer visited coordinates, fewer cache keys, fewer command branches, and cooperative service steps instead of blocking loops. The result is a smaller, more focused automation script that preserves the essential farming workflow while avoiding the largest known sources of wasted movement and runtime work.